Django Real-Time Chat App

Project Report

Built with Django, Django Channels, Daphne and WebSocket

Author	Muhammad Faizan Farooq
GitHub	github.com/faizanjutt473
Project	Project No. 4 (Django learning path)
Date	October 2026

1. Introduction

This project is a real-time chat application built with Django. Normal Django websites work on a request and response basis: the browser asks for a page and the server replies once. A chat app needs something different. When one user sends a message, every other user in the room must see it immediately, without refreshing the page.

To achieve this, the project uses **Django Channels**, which adds WebSocket support to Django. A WebSocket keeps one connection open between the browser and the server, so both sides can send data at any time. The project also includes user authentication, multiple chat rooms and a database that stores every message.

2. Objectives

- Build a chat application where messages appear instantly for all users in a room.
- Learn how Django Channels, consumers, routing and channel layers work.
- Store all messages in the database so the chat history is not lost.
- Add signup, login and logout, and allow only logged-in users to chat.
- Show the sender's username with every message in a clean chat interface.
- Create a portfolio-ready project that can be published on GitHub.

3. Technologies Used

Technology	Purpose
Python 3.11	Programming language
Django 5.2	Web framework: views, templates, ORM, authentication, admin
Django Channels	Adds WebSocket and async support to Django
Daphne	ASGI server that handles HTTP and WebSocket together
InMemoryChannelLayer	Lets consumers in the same room send messages to each other (development)
SQLite	Default Django database used to store users and messages
HTML, CSS, JavaScript	Frontend pages, chat design and the browser WebSocket client

4. System Architecture

The application has three main parts: the browser (client), the Django/Channels server and the database. Daphne receives all incoming traffic. **ProtocolTypeRouter** in `config/asgi.py` then decides where each request goes: normal HTTP requests go to Django (URLs, views and templates), and WebSocket connections go through **AuthMiddlewareStack** and **URLRouter** to the **ChatConsumer**.

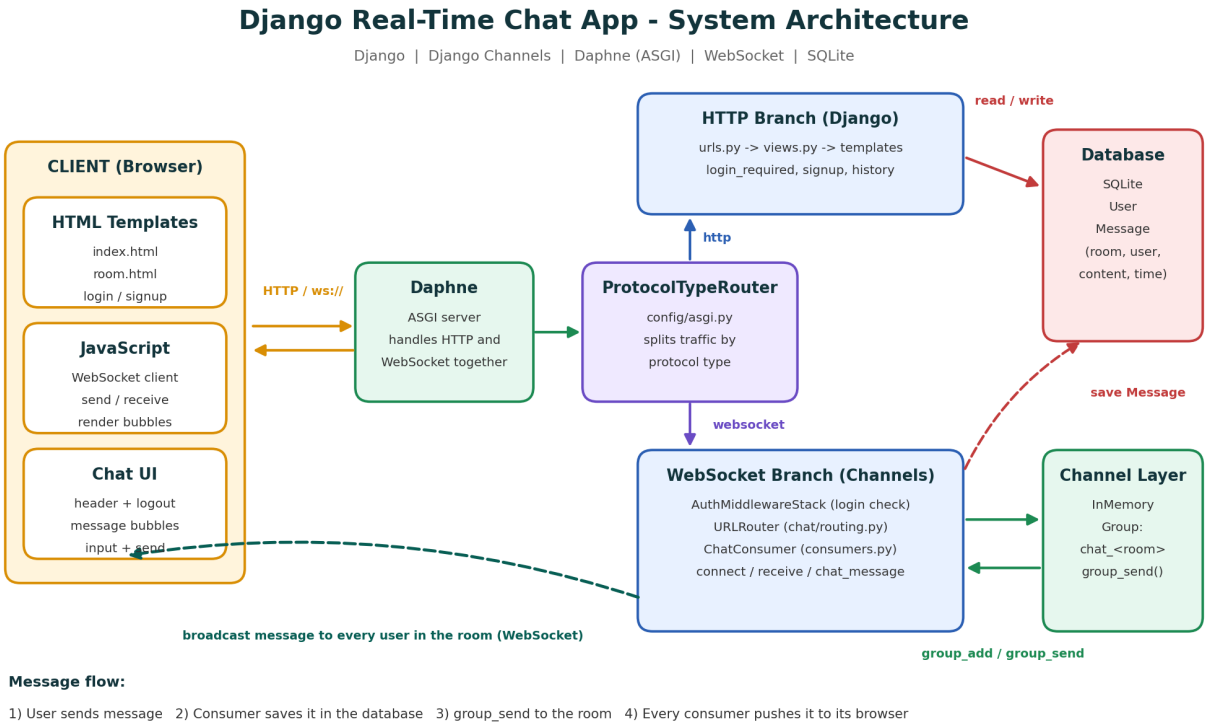


Figure 1: System architecture of the chat application

4.1 Message flow

- The user opens a room page. The browser creates a WebSocket connection to `/ws/chat/<room>/`.
- The consumer checks that the user is logged in. If not, the connection is closed.
- The consumer adds the connection to a channel group named `chat_<room>`.
- When the user sends a message, the consumer saves it in the database.
- The consumer calls `group_send` to broadcast the message to the whole group.
- Every consumer in the group receives the event and sends the message to its own browser.
- JavaScript adds a new message bubble to the chat window.

5. Database Design

The project uses Django's built-in **User** model and one custom model called **Message**. Each message belongs to a room and, optionally, to a user.

Field	Type	Description
room	CharField(255)	Name of the chat room
user	ForeignKey(User)	Sender of the message (null allowed so older messages stay valid)
content	TextField	The message text
timestamp	DateTimeField	Date and time, added automatically when the message is created

```
class Message(models.Model):
    room = models.CharField(max_length=255)
    user = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, blank=True)
    content = models.TextField()
    timestamp = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["timestamp"]
```

6. Implementation

6.1 Project modules

File	Role
config/settings.py	Registers daphne, channels and chat apps; sets ASGI_APPLICATION, CHANNEL_LAYERS and login redirects
config/asgi.py	ProtocolTypeRouter: HTTP to Django, WebSocket to Channels with AuthMiddlewareStack
config/urls.py	Admin, accounts (login, logout, signup) and chat URLs
chat/models.py	Message model
chat/views.py	index, room (with message history) and signup views; login_required protection
chat/routing.py	WebSocket route: ws/chat/<room_name>/
chat/consumers.py	ChatConsumer: connect, disconnect, receive, chat_message and save_message
chat/admin.py	Registers Message in the admin panel with list display, filter and search
templates/chat/*.html	Room selection page and the chat room page
templates/registration/*.html	Login and signup pages

6.2 The WebSocket consumer

The consumer is the heart of the application. It runs asynchronously, so one server process can handle many connected users at the same time. Because Django's database access is synchronous, saving a message is wrapped with `database_sync_to_async`.

```
async def connect(self):
    self.room_name = self.scope["url_route"]["kwargs"]["room_name"]
    self.room_group_name = f"chat_{self.room_name}"
    self.user = self.scope["user"]

    if not self.user.is_authenticated:
        await self.close()          # reject anonymous users
        return

    await self.channel_layer.group_add(self.room_group_name, self.channel_name)
    await self.accept()

async def receive(self, text_data):
    message = json.loads(text_data)["message"]
    await self.save_message(self.room_name, self.user, message)
    await self.channel_layer.group_send(
        self.room_group_name,
        {"type": "chat_message", "message": message,
         "username": self.user.username},
    )
```

6.3 Routing

Routing works like `urls.py`, but for WebSockets. The route below connects the address used by the browser to the consumer:

```
websocket_urlpatterns = [
    path("ws/chat/<str:room_name>/", consumers.ChatConsumer.as_asgi()),
]
```

6.4 Authentication

Django's built-in authentication views are used for login and logout, and a small custom view provides signup with `UserCreationForm`. The chat pages use `@login_required`, and the WebSocket consumer also checks `is_authenticated`, so a user cannot join a room by connecting to the WebSocket directly. Logout uses a POST form with a CSRF token, as required by Django 5.

6.5 Frontend

The chat page is built with plain HTML, CSS and JavaScript. When the page loads, the saved messages are passed from the view and displayed first. After that, the JavaScript WebSocket client appends every new message in real time. Messages sent by the current user appear as green bubbles on the right; messages from other users appear as grey bubbles on the left, each with the sender's username.

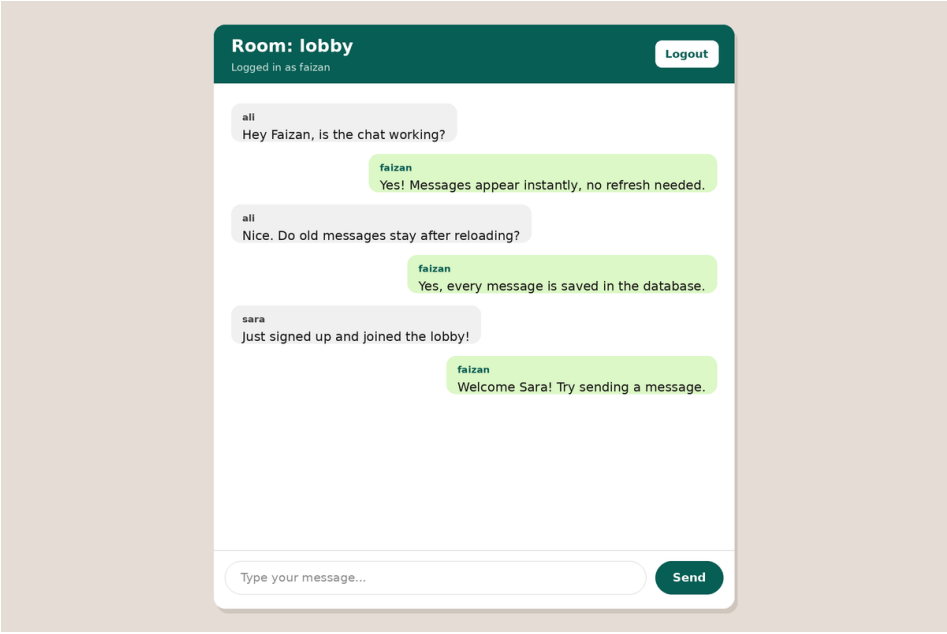


Figure 2: Chat room interface (illustrative preview)

7. Features Summary

- Instant messaging using WebSockets, without page refresh
- Unlimited chat rooms, created by simply entering a new room name
- Persistent chat history stored in the database
- Signup, login and logout
- WebSocket connections restricted to authenticated users
- Username shown with every message
- Message bubbles styled by sender (mine / others)
- Admin panel with message list, room filter and search

8. Testing

The application was tested manually in the browser. The table lists the main test scenarios and the behaviour that is expected from the application.

Scenario	Expected result
Run the server	Terminal shows Starting ASGI/Daphne version
Open /chat/lobby/ and send a message	WebSocket HANDSHAKING and CONNECT appear in the terminal; the message shows in the chat
Open the same room in two windows with two users	A message sent by one user appears instantly in both windows
Refresh the room page	Previous messages are loaded from the database
Open /chat/ while logged out	User is redirected to the login page
Log in with correct credentials	User is redirected to /chat/

Scenario	Expected result
Sign up with a new username	Account is created and the user is sent to the login page
Open the Messages section in /admin/	All saved messages are listed with room, user, content and time

9. Problems Faced and Solutions

Problem	Cause and solution
WebSocket request returned 404 (Not Found: /ws/chat/lobby/)	daphne was missing from INSTALLED_APPS, so the plain Django server was running. Adding daphne as the first app fixed it.
TemplateDoesNotExist for registration/login.html	The folder was named registeration. Renaming it to registration fixed it.
Pylance warnings in consumers.py	Type-checker warnings only (scope typing and a parameter name). Renaming close_code to code and adding a type: ignore comment cleaned them up.

10. Limitations and Future Improvements

- **Channel layer:** InMemoryChannelLayer works only inside one process. Production deployment should use channels-redis.
- **HTTPS:** on a secure website the WebSocket address must use wss:// instead of ws://.
- **Room names:** spaces and special characters are not handled yet and should be validated.
- **Message length and spam:** there is no limit or rate limiting yet.
- **Possible features:** online users list, typing indicator, message time, private one-to-one chats, file and image sharing, read receipts.

11. Conclusion

The project successfully demonstrates how to build a real-time application with Django. It covers the complete flow: WebSocket connection, authentication, message broadcasting, database storage and a friendly chat interface. It is a strong portfolio project because it goes beyond ordinary CRUD websites and shows understanding of asynchronous programming, ASGI and real-time communication.